Kolekcje współbieżne

by Paweł Paduch (https://achilles.tu.kielce.pl/portal/author/332bd2d158a24964b2b98a7fc2879842) published 7 years ago, last modified 6 years ago

Laboratorium, ma na celu zapoznanie się z kolekcjami współbieżnymi.

1. **Wstęp**
   Na dzisiejszych zajęciach poznamy mechanizmy kolekcji współbieżnych.

2. **Na początek, błędy (0,5)**
   Należy utworzyć solucję do dzisiejszego laboratorium a w niej projekt o nazwie *NiebezpiecznaKolekcja*.
   Wstaw poniższy kod i uruchom.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace NiebezpiecznaKolekcja
{
    class Program
    {
        static void Main(string[] args)
        {
            List<Thread> watki = new List<Thread>();
            List<int> liczby = new List<int>(1000000);
            Random rand = new Random();
            for (int i = 0; i < 100; i++)
            {
                var watek = new Thread(() =>
                {
                    for (int l = 0; l < 10000; l++)
                        liczby.Add(rand.Next());
                });
                watki.Add(watek);
                watek.Start();
            }
            foreach (var watek in watki)
            {
                watek.Join();
            }
            Console.WriteLine($"Liczba elementów w liście zwykłej: {liczby.Count}");
            Console.ReadLine();
        }
    }
}
```

   Powyższy kod tworzy listę 100 wątków. Każdy z wątków dodaje po 10000 losowych elementów do listy.
   Uruchomić kilka razy program. Ile elementów znajduje się w liście?
   Dodać pomiar czasu za pomocą *Stopwatch*.

3. **ConcurrentBag (0,5)**
   Zamiast zwykłej listy zastosować kolekcję *ConcurrentBag* i zmierzyć czas od startowania wątków do ich zakończenia.
4. **Synchronizacja za pomocą *lock* (0,5)**
   Do pierwszego programu (z punktu 2) zastosować synchronizację za pomocą *locka*. Także zmierzyć czas wykonania.

5. **Unikalna kolekcja (0,5)**
   - Dodać nowy projekt o nazwie "KolekcjaUnikalna"
   - Dodać klasę Extensions

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace KolekcjaUnikalnaLock
{
    public static class Extensions
    {

        public static bool IsUnique(this List<int> lista)
        {
            if (lista.Distinct().Count() == lista.Count()) //jeżeli liczba unikalnyc
            {
                return true;
            }
         //   Console.WriteLine($"Liczba disctinct: {lista.Distinct().Count()}  jest
            return false;
        }

        public static bool IsUnique(this ConcurrentBag<int> cb)
        {
            if (cb.Distinct().Count() == cb.Count()) return true;
            else return false;
        }
    }
}
```

   - Wstawić kod Main:

```
List<Thread> watki = new List<Thread>();
ConcurrentBag<int> liczby = new ConcurrentBag<int>();
Stopwatch sw = new Stopwatch();
sw.Start();
Random rand = new Random();
for (int i = 0; i < 100; i++)
{
    var watek = new Thread(() =>
    {

        for (int l = 0; l < 100; l++)
            {
                int liczba = rand.Next(10001);
                while (liczby.Any(x => x == liczba))
                {
                    liczba = rand.Next(10001);
                };
                liczby.Add(liczba);
            }
    });
    watki.Add(watek);
    watek.Start();
}
foreach (var watek in watki)
{
    watek.Join();
}
sw.Stop();

Console.WriteLine($"Upłynęło {sw.ElapsedMilliseconds} ms");
if (liczby.IsUnique()) //zastosowanie metody rozszerzającej dla CB
    Console.WriteLine("wszystkie liczby są unikalne");
else
    Console.WriteLine("Nie wszystkie liczby są unikalne");


Console.ReadLine();
```

- Przykładowy kod nie zapewnia unikalnych wartości. Za pomocą lock zmodyfikuj kod by wartości były unikalne.

6. **ConcurrentDictionary (0,5)**
   - Dodaj nowy projekt "KolekcjaDictionary".
   - Bazując na powyższym użyj *ConcurrentDictionary* do zapewnienia unikalności kolekcji losowanych liczb.
   - Sprawdzić czy wszystkie elementy się dodały (dla powyższego przykładu powinno być 10000)
   - Porównaj z czasami uzyskanymi w poprzednich przykładach.

7. **Producent i konsument oparty na kolejce (1,5)**
   - Stworzyć nowy projekt *ProducentKonsumentQ*
   - Użyć poniższego kodu i zobaczyć efekt

```
var cq = new Queue<int>();
          List<Thread> watki = new List<Thread>();
          Random rand = new Random();
          //prodcent wkłada do kolekcji 100 elementów z losowymi odstępami do 500m
          var producent = new Thread(() =>
          {
              for (int l = 0; l < 100; l++)
              {
                  cq.Enqueue(l);
                  Console.WriteLine($"Wkładam  {l}");
                  Thread.Sleep(rand.Next(500));
              }

          });
          producent.Start();

          for (int i = 0; i < 10; i++)
          {
              var watek = new Thread(() =>
              {

                  for (int l = 0; l < 10; l++)
                  {
                      try
                      {
                          int result2 = cq.Dequeue();
                          Console.WriteLine($"Odbieram {result2} ");
                      }
                      catch (Exception ex)
                      {
                          Console.WriteLine("Wyjątek przy odebraniu: ", ex.Message
                      }

                      Thread.Sleep(rand.Next(5000));
                  }
              });
              watki.Add(watek);
              watek.Start();
          }
          foreach (var watek in watki)
          {
              watek.Join();
          }
          producent.Join();
```

- Przerobić program, by używał mechanizmu *ConcurrentQueue*
- W przypadku ConcurrentQueue brak blokującej metody *Dequeue*, za to jest metoda próbująca odczytać dane:

  ```
  bool udaloSie = cq.TryDequeue(out result2);
  ```

  Można badać stan i w razie niepowodzenia powtórzyć operację tak, by każdy wątek odczytał swoją porcję danych. Jeżeli tego nie zrobimy wątki czytające skończą się a w kolejce pozostaną nieodczytane dane. (0,5).
- Zamiast badać stan odczytu, użyć semafora do synchronizacji tak, by wszystkie elementy zostały odczytane. (0,5)

8. **Producent konsument na *BlockingCollection* (1)**
    - Wykorzystując powyższy przykład, przerobić go tak by wykorzystał on kolekcję blokującą
    - kolekcja blokująca powinna mieć pojemność 10 elementów
    - po upływie 5s od wystartowania producenta wystartować wątki konsumentów
    - wątki konsumentów powinny 10 razy pobrać liczbę z kolekcji w losowych odstępach max 5s.
    - Przy wkładaniu i pobieraniu danych wypisać na konsole informacje o tym fakcie.

9. Proszę nie zapomnieć o sprawozdaniu!